

# Архитектурный проект и инженерная декомпозиция: Интеллектуальный офлайн-редактор фотографий для ОС Аврора (с использованием ONNX Runtime)

---

Разработка современного программного обеспечения для мобильных платформ, ориентированного на интенсивную обработку графики и использование алгоритмов машинного обучения (Machine Learning, ML), требует глубокого понимания как аппаратной архитектуры, так и системного программного интерфейса. В контексте операционной системы Аврора, которая представляет собой POSIX-совместимый дистрибутив Linux, адаптированный для корпоративного и государственного секторов, на первый план выходят вопросы безопасности, приватности и оптимизации энергопотребления. Идея создания ИИ-редактора фотографий, функционирующего полностью в офлайн-режиме, идеально резонирует с парадигмой On-Device Machine Learning (ODML). Данный подход гарантирует нулевую задержку, связанную с сетевыми запросами, обеспечивает строгую приватность данных пользователя, снижает потребление энергии радиомодулями и позволяет уменьшить размер развертываемых бинарных файлов за счет использования квантованных моделей.

ОС Аврора исторически опирается на связку C++ для реализации высокопроизводительной бизнес-логики и декларативного языка QML для создания адаптивных пользовательских интерфейсов. Интеграция этих двух слоев осуществляется через мета-объектную систему фреймворка Qt. Подобная архитектура диктует строгие правила изоляции: тяжелые вычисления, такие как инференс нейронных сетей и матричные преобразования изображений, не должны блокировать главный поток отрисовки пользовательского интерфейса (UI). Любое провисание главного потока приведет к деградации пользовательского опыта и может быть расценено операционной системой как зависание, что повлечет принудительное завершение процесса (OOM-killer или системный watchdog).

Для реализации заявленного функционала — вырезания и замены фона, интеллектуального улучшения кадра, художественной стилизации и конвейерной обработки — требуется внедрение специализированных библиотек. Использование **ONNX Runtime (ORT)** в качестве среды исполнения моделей машинного обучения обеспечивает высокую кроссплатформенную переносимость, богатую поддержку операторов нейросетей и высокую производительность на ARM-архитектурах за счет встроенного **CPU Execution Provider** (с поддержкой оптимизированных под ядра ARM библиотек MLAS/NEON) и возможности подключения **XNNPACK Execution Provider**. Для классического препроцессинга и рутинных операций компьютерного зрения целесообразно

интегрировать ядро библиотеки **OpenCV**, что позволит сократить время разработки алгоритмов цветокоррекции и масштабирования.

---

## Распределение компетенций и методология разработки

---

Для успешной и ритмичной реализации проекта командой из шести инженеров необходимо четко разграничить зоны ответственности. Учитывая монолитный характер целевого бинарного файла, но модульную внутреннюю структуру, команда разделяется на специалистов по пользовательскому интерфейсу (QML/UI), специалистов по интеграции машинного обучения и системных архитекторов C++.

| Роль в команде          | Инженер       | Зона ответственности и фокус предметной области                                                                                                                                                                                                                                    |
|-------------------------|---------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Ведущий C++ архитектор  | Разработчик 1 | Проектирование паттерна Command для конвейера обработки, управление памятью (отсутствие утечек QImage), реализация PipelineManager и координация потокобезопасного доступа к данным.                                                                                               |
| ML & C++ Инженер        | Разработчик 2 | Интеграция <b>ONNX Runtime (ORT)</b> , управление жизненным циклом среды исполнения (Ort::Env), сессий (Ort::Session) и тензоров (Ort::Value), квантование ONNX моделей, написание оберток MLInferenceEngine и делегирование вычислений на ядра ARM с оптимизацией (NEON/XNNPACK). |
| Инженер Computer Vision | Разработчик 3 | Разработка алгоритмов препроцессинга (масштабирование), постпроцессинга (alpha blending, edge refinement) и алгоритмов классической автокоррекции (CLAHE, Gray World).                                                                                                             |

| Роль в команде             | Инженер       | Зона ответственности и фокус предметной области                                                                                                                                                                        |
|----------------------------|---------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Ведущий QML/UI Разработчик | Разработчик 4 | Интеграция C++ и QML через <code>QQuickImageProvider</code> , разработка главного рабочего экрана, управление состояниями интерфейса и навигацией.                                                                     |
| QML/UX Разработчик         | Разработчик 5 | Разработка кастомных контролов (слайдеры, палитры), интеграция системных компонентов <code>Sailfish.Pickers</code> , реализация индикаторов загрузки и анимаций.                                                       |
| DevOps и Системный Инженер | Разработчик 6 | Настройка сборочного конвейера (CMake/qmake), написание <code>.spec</code> файлов для RPM, управление пулом потоков <code>QThreadPool</code> , безопасное прерывание задач, профилирование (Valgrind) и разрешения ОС. |

Процесс разработки декомпозирован на логические слои (Layers). Каждый последующий слой опирается на стабильный фундамент предыдущего. Подобная архитектура позволяет минимизировать блокировки (bottlenecks) между членами команды и обеспечивает возможность непрерывного тестирования (Continuous Testing) по мере добавления нового функционала. Ниже представлено детальное техническое описание каждого слоя с соответствующим списком задач, готовым для импорта в систему отслеживания ошибок (Issue Tracker).

---

## Слой 0: Фундаментальная инфраструктура, сборка и изоляция

---

Нулевой слой закладывает основу всего продукта. На этом этапе формируется скелет приложения, настраивается система сборки, интегрируются внешние зависимости и определяются политики безопасности в среде ОС Аврора. Приложения в ОС Аврора функционируют в условиях песочницы (sandbox), что требует явного декларирования всех необходимых разрешений в

файле конфигурации. Поскольку ключевой ценностью продукта является приватность, сетевые разрешения (Network) полностью исключаются. Приложению требуется исключительно локальный доступ к медиафайлам пользователя.

Интеграция сторонних библиотек, таких как **ONNX Runtime** и **OpenCV**, в сборочный конвейер Авроры может осуществляться через `qmake` или `CMake`. `CMake` становится предпочтительным выбором для сложных проектов благодаря развитой системе модулей. Библиотека ONNX Runtime (заголовочные файлы C++ API и динамическая библиотека `libonnxruntime.so` / статические библиотеки) должна быть собрана под целевые архитектуры (`armv7hl` и `aarch64`) с включенными флагами оптимизации для ARM-процессоров (поддержка NEON и XNNPACK Execution Provider).

Оптимизация памяти на этом этапе имеет критическое значение. В отличие от TFLite, где переиспользуется Interpreter, в ONNX Runtime ключевыми тяжелыми объектами являются `Ort::Env` (окружение среды исполнения, создается один раз на все приложение) и `Ort::Session` (сессия инференса конкретной модели). Необходимо спроектировать синглтон-менеджер, который будет инициализировать `Ort::Env`, а также загружать веса моделей и создавать `Ort::Session` в оперативной памяти только один раз при старте или по требованию. Для избежания постоянных аллокаций памяти во время инференса, входные и выходные тензоры (`Ort::Value`) должны использовать предварительно выделенные буферы памяти (memory arena), предоставляемые ORT (`Ort::SessionOptions::EnableCpuMemArena()`).

Архитектурным ядром приложения станет паттерн **"Команда" (Command)**. В рамках этого паттерна каждое действие пользователя (применение фильтра, изменение яркости, стилизация) инкапсулируется в объект, содержащий методы `execute()` и `undo()`. Менеджер конвейера (`PipelineManager`) будет управлять стеком этих команд. Для обеспечения функциональности сброса (reset) оригинал фотографии должен храниться в памяти в неизменном виде, тогда как все операции применяются к рабочей копии (`workingCopy`).

Задачи Слая 0

| Идентификатор задачи | Название и описание архитектурной задачи                                            | Исполнитель   | Зависимости | Критерии приемки (DoD)                                                                                            |
|----------------------|-------------------------------------------------------------------------------------|---------------|-------------|-------------------------------------------------------------------------------------------------------------------|
| ISSUE-0.1            | Инициализация проекта и настройка CI/CD (CMake/qmake)<br>Создание базовой структуры | Разработчик 6 | Нет         | Успешная компиляция пустого приложения в Aurora SDK.<br>Сборка RPM пакета проходит без ошибок. Автоматизированный |

| Идентификатор задачи | Название и описание архитектурной задачи                                                                                                                                                                                                                                                                        | Исполнитель   | Зависимости | Критерии приемки (DoD)                                                                                                                                  |
|----------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------|-------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|
|                      | директорий (src, qml, rpm).<br>Написание main.cpp с загрузкой движка QCoreApplicationEngine.<br>Настройка .spec файла для упаковки в RPM пакет.                                                                                                                                                                 |               |             | запуск тестов базовых классов проходит успешно.                                                                                                         |
| ISSUE-0.2            | <b>Конфигурация изоляции и разрешений (Permissions)</b><br>В файле .desktop необходимо настроить раздел [X-Application]. Для работы с фотогалереей требуется указать Permissions=Media;Pictures. Запретить сетевой доступ на уровне манифеста.                                                                  | Разработчик 6 | ISSUE-0.1   | Манифест приложения проходит системную валидацию. Приложение имеет доступ к директории ~/Pictures, но попытки системных сетевых вызовов блокируются ОС. |
| ISSUE-0.3            | <b>Интеграция ONNX Runtime и OpenCV в сборочную систему</b><br>Добавление заголовочных файлов и библиотек ONNX Runtime (C++ API) и opencv_core/opencv_imgcodecs. Написание базового C++ класса MLInferenceEngine для инициализации глобального Ort::Env и создания сессий Ort::Session из файлов моделей .onnx. | Разработчик 2 | ISSUE-0.1   | В C++ коде успешно инициализируются Ort::Env и Ort::Session. Проект компилируется под armv7hl/aarch64 без ошибок линковки (undefined references).       |
| ISSUE-0.4            | <b>Реализация паттерна Command и PipelineManager</b><br>Создание интерфейса ImageEditorCommand и класса                                                                                                                                                                                                         | Разработчик 1 | ISSUE-0.1   | Наличие методов applyCommand(), undoLast(), resetToOriginal(). Покрывание unit-тестами логики стека команд. Использование                               |

| Идентификатор задачи | Название и описание архитектурной задачи                                                                                                            | Исполнитель | Зависимости | Критерии приемки (DoD)                             |
|----------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|-------------|-------------|----------------------------------------------------|
|                      | PipelineManager<br>(унаследованного от QObject).<br>Реализация хранения оригинала (QImage original) и отдельной рабочей копии (QImage workingCopy). |             |             | QMutex для защиты данных при конкурентном доступе. |

## Слой 1: Пользовательский интерфейс, ввод/вывод и передача изображений

Данный слой посвящен созданию моста между пользовательским интерфейсом на QML и тяжелой бизнес-логикой на C++. ОС Аврора предоставляет богатый набор стандартных компонентов, таких как Aurora.Controls и Sailfish.Silica. Для выбора изображений из файловой системы устройства необходимо использовать модуль Sailfish.Pickers, который предоставляет нативные и безопасные диалоги.

Ключевой технической проблемой на данном этапе является эффективное отображение изменяемой в C++ памяти фотографии на экране QML без промежуточного сохранения на диск. Для решения этой задачи фреймворк Qt предлагает класс QQuickImageProvider. Этот класс позволяет QML запрашивать изображения по специальному URL-протоколу (например, image://editor/working\_copy). Провайдер перехватывает этот запрос, обращается к PipelineManager и возвращает актуальный QImage.

Однако движок QML агрессивно кэширует изображения для оптимизации производительности. Если URL не меняется, QML не будет вызывать метод requestImage() провайдера. Для обхода этого механизма и обеспечения мгновенного визуального отклика после применения фильтров, в QML необходимо динамически добавлять уникальный идентификатор (например, метку времени Unix) к URL при каждом изменении, заставляя движок инвалидировать кэш и запрашивать свежий кадр: source: "image://editor/working\_copy?" + layoutTimestamp

Также следует учитывать известные особенности отображения изображений. В некоторых версиях систем на базе Sailfish/Aurora OS наблюдался баг, при котором изображения в компонентах Canvas или при передаче через ImageProvider могли получать синий

оттенков из-за неправильной интерпретации цветовых каналов (смена каналов Red и Blue) или неверной обработки метаданных EXIF об ориентации. С++ ядро должно нормализовать загружаемые изображения, приводя их к формату QImage::Format\_RGBA8888 или QImage::Format\_RGB32, а также применять матрицу трансформации на основе EXIF-данных до передачи в конвейер обработки.

Задачи Слой 1

| Идентификатор задачи | Название и описание архитектурной задачи                                                                                                                                                                                                                              | Исполнитель   | Зависимости          | Критерии приемки (DoD)                                                                                                                                                                  |
|----------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------|----------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ISSUE-1.1            | <b>Интеграция Sailfish.Pickers для импорта фото</b><br>Создание страницы QML с вызовом диалога ImagePickerPage. Обработка полученного URI, чтение файла в С++, учет EXIF-ориентации и конвертация в нормализованный RGB-формат для предотвращения цветовых искажений. | Разработчик 5 | ISSUE-0.2            | Полноэкранный выбор фото. Изображение корректно загружается в PipelineManager. Отсутствуют искажения пропорций, цвета (blue tint) и поворота.                                           |
| ISSUE-1.2            | <b>Реализация QQuickImageProvider для холста</b><br>Написание класса-наследника QQuickImageProvider. Переопределение метода requestImage. Регистрация провайдера в QQmlApplicationEngine с помощью engine.addImageProvider().                                         | Разработчик 4 | ISSUE-0.4, ISSUE-1.1 | QML элемент Image успешно запрашивает данные по протоколу image://. Обновление параметра source с timestamp заставляет UI мгновенно перерисовать измененную картинку без утечек памяти. |
| ISSUE-1.3            | <b>Экспорт результата (сохранение в исходном разрешении)</b>                                                                                                                                                                                                          | Разработчик 3 | ISSUE-0.4            | Файл сохраняется без блокировки UI. Появляется системное уведомление                                                                                                                    |

| Идентификатор задачи | Название и описание архитектурной задачи                                                                                                                                                                                              | Исполнитель   | Зависимости | Критерии приемки (DoD)                                                                                                                                                                     |
|----------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------|-------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                      | Асинхронная запись итогового QImage в директорию <code>QStandardPaths::PicturesLocation</code> . Генерация уникального имени файла с таймстампом. Установка степени сжатия JPEG на уровне 95%.                                        |               |             | ( <code>NoticePage.qml</code> ) об успехе. Исходное разрешение (например, 12 Мп) сохраняется полностью.                                                                                    |
| ISSUE-1.4            | <b>Верстка базового QML-каркаса редактора</b><br>Создание главного окна приложения с холстом для фото и нижними панелями инструментов («Фон», «Улучшение», «Стиль»).<br>Создание верхнего AppBar с кнопками Отмена, Сброс, Сохранить. | Разработчик 4 | ISSUE-1.2   | Интерфейс соответствует гайдлайнам Авроры. Панели корректно скрываются/появляются с анимацией. Кнопка «Отмена» привязана к C++ методу <code>undoLast()</code> и неактивна, если стек пуст. |

---

## Слой 2: Асинхронные вычисления, масштабирование и управление ресурсами

---

Обработка полноразмерных фотографий (от 12 до 40 мегапикселей) на мобильном устройстве является чрезвычайно ресурсоемкой задачей. Если выполнять такие операции в основном потоке приложения, операционная система зафиксирует отсутствие отклика пользовательского интерфейса (Application Not Responding) и принудительно завершит процесс. Для соответствия нефункциональным требованиям все тяжелые операции, включая препроцессинг и инференс моделей, должны выполняться в фоновых потоках с возможностью безопасной отмены.

В Qt для этих целей используется пул потоков `QThreadPool` в связке с интерфейсом `QRunnable`. Каждый инструмент редактора (сегментация, улучшение, стилизация) должен быть оформлен как отдельная асинхронная задача. Для обеспечения возможности



прерывания (например, если пользователь нажал кнопку «Отмена» во время обработки) каждая задача должна в циклах обработки пикселей или между шагами подготовки данных проверять атомарный флаг прерывания (`std::atomic<bool>`) или использовать метод `QThread::currentThread()->isInterruptionRequested()`.

Жизненный цикл фоновых потоков также критически важен при завершении работы приложения: если пользователь смахивает приложение во время работы нейросети, процесс должен корректно перехватить сигнал закрытия, отправить запрос на остановку всем потокам, дождаться их завершения (`graceful shutdown`) и освободить память, избегая аварийных дампов (`segmentation faults`).

Еще одной фундаментальной проблемой является размер входных данных для нейронных сетей. Нейросети имеют фиксированный размер входного тензора (например, 256x256 или 512x512 пикселей). Подача 12-мегапиксельного изображения напрямую в модель невозможна. Следовательно, конвейер обработки должен включать этап понижающего масштабирования (`downscaling`) оригинального изображения до фиксированного размера перед инференсом.

После того как нейросеть вернет результат (например, маску сегментации размером 256x256), этот результат должен быть масштабирован обратно (`upscaling`) до исходного разрешения фотографии с использованием билинейной или бикубической интерполяции, и только затем применен к оригинальному изображению.

Для оптимизации инференса в ONNX Runtime на мобильных устройствах необходимо настроить сессию (`Ort::SessionOptions`): 1. **Thread Pool Configuration:** Задать оптимальное количество потоков для инференса одной модели. На мобильных ARM-процессорах с архитектурой big.LITTLE оптимально ограничить intra-оп параллелизм количеством быстрых ядер (обычно от 2 до 4 потоков) с помощью `SetIntraOpNumThreads()`, чтобы избежать избыточного переключения контекста. 2. **Memory Arena:** Включить оптимизацию пула памяти через `EnableCpuMemArena()` для минимизации фрагментации ОЗУ. 3. **Execution Providers:** Явно настроить приоритет провайдеров выполнения. При сборке с поддержкой XNNPACK добавить его инициализацию в опции сессии для ускорения выполнения стандартных сверточных слоев на CPU.

Для прозрачности процесса и оптимизации необходимо интегрировать систему замеров времени выполнения каждого этапа (препроцессинг, инференс ORT, постпроцессинг) с использованием `QElapsedTimer`.

## Задачи Слой 2

| Идентификатор задачи | Название и описание архитектурной задачи                                                                                                                                                                                                                                  | Исполнитель   | Зависимости | Критерии приемки (DoD)                                                                                                                                                    |
|----------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------|-------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ISSUE-2.1            | <b>Разработка Background Task Manager</b><br>Создание базового класса BackgroundWorker (наследник QRunnable и QObject для сигналов). Реализация сигналов прогресса, успеха и отмены. Настройка QThreadPool на оптимальное количество потоков.                             | Разработчик 6 | ISSUE-0.4   | Операции выполняются в фоне. UI остается отзывчивым. Вызов метода cancel() немедленно прерывает выполнение, рабочая копия QImage остается нетронутой.                     |
| ISSUE-2.2            | <b>Безопасное завершение потоков (Graceful Shutdown)</b><br>Перехват системного события закрытия приложения. Реализация механизма waitForDone(3000) в деструкторе основного контроллера. Принудительное прерывание всех флагов isInterruptionRequested().                 | Разработчик 6 | ISSUE-2.1   | Отсутствие крэшей и зависаний в ядре Linux при резком закрытии приложения пользователем во время активной фазы работы нейросети. Проверка через утилиту top / journalctl. |
| ISSUE-2.3            | <b>Модуль масштабирования (Downscale / Upscale)</b><br>Реализация C++ утилит для быстрого ресайза больших изображений (используя быстрые методы интерполяции OpenCV или Qt::FastTransformation) до размеров тензора, и качественного сглаженного апскейла масок или слоев | Разработчик 3 | ISSUE-0.3   | Функции prepareModelInput и upscaleResult написаны и оптимизированы. Замеры показывают время препроцессинга не более 50-150 мс для 12 Мп фото на архитектуре ARM.         |

| Идентификатор задачи | Название и описание архитектурной задачи                                                                                                                                                                                                                                      | Исполнитель   | Зависимости | Критерии приемки (DoD)                                                                                                                               |
|----------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------|-------------|------------------------------------------------------------------------------------------------------------------------------------------------------|
|                      | (Qt::SmoothTransformation) до оригинала.                                                                                                                                                                                                                                      |               |             |                                                                                                                                                      |
| ISSUE-2.4            | <b>Система профилирования и замеров времени</b><br>Интеграция микросекундных таймеров в каждый BackgroundWorker. Логирование времени, затраченного на препроцессинг, передачу данных в ONNX Runtime, непосредственно инференс (Ort::Session::Run) и постпроцессинг (апскейл). | Разработчик 2 | ISSUE-2.1   | Вывод структурированных логов в консоль приложения. Наличие возможности программно агрегировать эти данные для выявления «узких мест» в архитектуре. |

---

### Слой 3: Инструмент «Улучшение» (Алгоритмическая автокоррекция)

---

Функциональное требование к инструменту «Улучшение» предполагает автоматическую коррекцию яркости, контраста и баланса белого. Хотя эту задачу можно решить с помощью сложных нейронных сетей типа Auto-Encoder, использование тяжелого инференса здесь не оправдано ни с точки зрения производительности, ни с точки зрения потребления батареи. Классические алгоритмы компьютерного зрения обеспечивают высочайшее качество и выполняются в десятки раз быстрее.

Для выравнивания локального контраста и яркости будет использоваться алгоритм **CLAHE** (Contrast Limited Adaptive Histogram Equalization). В отличие от глобального выравнивания гистограммы, CLAHE разбивает изображение на небольшие блоки (тайлы) и выравнивает гистограмму внутри каждого из них, предотвращая появление неестественных засветов на уже ярких участках. Для обработки цвета изображение предварительно конвертируется из цветового пространства RGB в пространство LAB (или

YCrCb), где канал светлоты (L) обрабатывается алгоритмом CLAHE, а цветовые каналы (A и B) остаются неизменными, что исключает искажение цветопередачи.

Для автоматического баланса белого применяется алгоритм **«Серого мира»** (Gray World Assumption), который базируется на предположении, что средний цвет любой естественной сцены является нейтрально-серым. Если средний цвет изображения имеет сдвиг (например, в желтые оттенки при свете ламп накаливания), алгоритм вычисляет поправочные коэффициенты для каждого канала и нейтрализует цветовой сдвиг.

Эти алгоритмы реализуются с использованием ядра библиотеки OpenCV. Оптимизация предполагает использование пулов памяти для матриц `cv::Mat`, чтобы избежать динамической аллокации (`malloc/free`) при каждой итерации, что существенно ускоряет работу программы на мобильных системах.

Задачи Слой 3

| Идентификатор задачи | Название и описание архитектурной задачи                                                                                                                                                                                                               | Исполнитель   | Зависимости          | Критерии приемки (DoD)                                                                                                                                                          |
|----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------|----------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ISSUE-3.1            | <b>Алгоритмы автокоррекции (CLAHE &amp; Gray World) в C++</b><br>Написание EnhanceCommand (наследника ImageEditorCommand).<br>Конвертация QImage в cv::Mat.<br>Перевод в LAB, применение CLAHE к L-каналу, расчет средних значений для баланса белого. | Разработчик 3 | ISSUE-0.4, ISSUE-2.1 | Алгоритм корректно вытягивает детали из теней и убирает локальные пересветы. Цветовая температура выравнивается. Время работы в фоновом потоке не превышает 2 секунд для 12 Мп. |
| ISSUE-3.2            | <b>Интеграция инструмента "Улучшение" в QML</b><br>Добавление кнопки (иконка "Волшебная палочка") на панель инструментов. Привязка события onClicked к вызову асинхронного                                                                             | Разработчик 5 | ISSUE-1.4, ISSUE-3.1 | Пользователь видит индикацию прогресса. Интерфейс не блокируется. По завершении QQuickImageProvider обновляет холст. Кнопка «Отмена» возвращает оригинал.                       |

| Идентификатор задачи | Название и описание архитектурной задачи                              | Исполнитель | Зависимости | Критерии приемки (DoD) |
|----------------------|-----------------------------------------------------------------------|-------------|-------------|------------------------|
|                      | метода в C++. Отображение <code>BusyIndicator</code> во время работы. |             |             |                        |

## Слой 4: Инструмент «Фон» (Нейросетевая сегментация переднего плана)

Замена фона является наиболее технологически сложной задачей в продукте. Она требует точного понимания семантики изображения (где находится человек или объект, а где фон) и сложной математики смешивания пикселей. Решение базируется на использовании предобученной легковесной нейронной сети для сегментации (например, **MediaPipe Selfie Segmentation** или **MobileNetV3-DeepLab**), сконвертированной в формат ONNX. Модель на выходе генерирует одноканальное изображение (маску), где каждый пиксель содержит вероятность (от 0.0 до 1.0) принадлежности к переднему плану.

Процесс обработки, оформленный в виде команды `BackgroundReplaceCommand`, начинается с подготовки тензора: 1. Масштабирование исходного изображения до входного разрешения модели (например, 256x256 или 512x512). 2. Преобразование пикселей в тензор с плавающей точкой (float32) и нормализация значений (приведение к диапазону [0.0, 1.0] или [-1.0, 1.0] в соответствии с требованиями модели). 3. Создание входного тензора `Ort::Value::CreateTensor` с использованием подготовленного непрерывного буфера памяти. 4. Вызов `Ort::Session::Run` в фоновом потоке. 5. Получение выходного тензора, содержащего маску вероятностей.

После получения матрицы 256x256, содержащей альфа-значения маски, эта матрица проходит через фильтр Гаусса для сглаживания границ (предотвращение эффекта «лесенки» или алиасинга) и масштабируется обратно до разрешения исходного изображения (до 12–40 мегапикселей).

Самым ресурсоемким этапом является **Alpha Blending** — попиксельное наложение исходного изображения на новый фон с использованием вычисленной альфа-маски. Формула наложения для каждого цветового канала выглядит как:  $\text{Result} = \text{Foreground} * \text{Alpha} + \text{Background} * (1.0 - \text{Alpha})$

Проход по десяткам миллионов пикселей в однопоточном режиме займет слишком много времени. Поэтому алгоритм блендинга должен быть распараллелен с использованием директив `OpenMP` или `QtConcurrent::map`, разбивая изображение на блоки (chunks), каждый из которых обрабатывается на отдельном ядре процессора ARM.

Важной архитектурной оптимизацией является кэширование апскейл-маски в памяти: если пользователь просто меняет цвет нового фона или интенсивность его размытия в UI, повторный инференс нейросети не запускается. Применяется только алгоритм Alpha Blending с закэшированной маской, что обеспечивает практически мгновенный отклик интерфейса (в пределах 60 FPS).

Задачи Слой 4

| Идентификатор задачи | Название и описание архитектурной задачи                                                                                                                                                                                                                                       | Исполнитель   | Зависимости | Критерии приемки (DoD)                                                                                                                                                   |
|----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------|-------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ISSUE-4.1            | <b>Интеграция ONNX-модели сегментации (Inference)</b><br>Загрузка .onnx файла модели.<br>Заполнение входного тензора Ort::Value пикселями уменьшенной копии.<br>Выполнение инференса через Ort::Session::Run в фоне.<br>Извлечение вероятностной карты маски размером 256x256. | Разработчик 2 | ISSUE-2.3   | Модель успешно распознает контуры людей/объектов на тестовых фото. Перерасход памяти при повторном инференсе отсутствует (сессия и тензоры Ort::Value переиспользуются). |
| ISSUE-4.2            | <b>Постобработка маски и многопоточный Alpha Blending</b><br>Апскейл маски до оригинального размера. Применение размытия (Blur) к краям маски для сглаживания. Написание многопоточной функции попиксельного смешивания двух                                                   | Разработчик 3 | ISSUE-4.1   | Отсутствие грубых пиксельных краев у вырезанных объектов. Скорость Alpha Blending для 12 Мп фото не превышает 300-500 мс благодаря распараллеливанию.                    |

| Идентификатор задачи | Название и описание архитектурной задачи                                                                                                                                                                                                                                     | Исполнитель                                      | Зависимости          | Критерии приемки (DoD)                                                                                                                                                   |
|----------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------|----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                      | QImage на базе полученной маски (OpenMP/QtConcurrent).                                                                                                                                                                                                                       |                                                  |                      |                                                                                                                                                                          |
| ISSUE-4.3            | <b>Разработка QML-панели управления фоном и кэширование</b><br>Создание контролов для выбора цвета, градиента и ползунок размытия фона. Реализация кэширования маски в C++, чтобы изменение настроек фона вызывало только пересчет Alpha Blending, минуя повторный инференс. | Разработчик 5 (UI) + Разработчик 1 (Архитектура) | ISSUE-1.4, ISSUE-4.2 | При движении ползунка размытия фона QML-интерфейс обновляется плавно, без рывков. Полный пересчет занимает доли секунды. Вкладки «Цвет» / «Размытие» работают корректно. |

## Слой 5: Инструмент «Стиль» (Нейросетевой перенос стиля)

Художественная стилизация изображения реализуется с помощью алгоритмов Neural Style Transfer (NST), которые извлекают семантическое содержимое одной фотографии и накладывают на нее текстурные и цветовые паттерны картины (например, произведений Ван Гога, Мунка или Пикассо). Модели NST (доступные в формате ONNX) принимают на вход изображение фиксированного размера (обычно 256x256 или 384x384 пикселя) и выдают стилизованное изображение такого же разрешения.

Здесь возникает прямое противоречие с функциональным требованием сохранения исходного разрешения: стилизованный результат 384x384 не может быть просто растянут до 12 мегапикселей, так как это приведет к катастрофической потере детализации и появлению пиксельных блоков. Для решения этой проблемы применяется метод композитинга с сохранением ярких деталей: 1. Оригинальное полноразмерное изображение разбивается на низкочастотные (цвет, крупные формы) и высокочастотные (мелкие детали, границы объектов) компоненты с помощью пространственных фильтров (например, размытия

Гаусса или двустороннего фильтра). 2. Стилизованное изображение малого разрешения, полученное на выходе сессии ONNX Runtime, масштабируется билинейной интерполяцией до исходного размера и используется в качестве базового цветового слоя (низкочастотного). 3. Поверх этого слоя в режиме наложения **Luminosity** (Яркость) накладываются высокочастотные детали оригинального снимка. Таким образом, фотография приобретает живописный стиль, но не теряет своей четкости и исходного разрешения.

Задачи Слая 5

| Идентификатор задачи | Название и описание архитектурной задачи                                                                                                                                                                                                                                               | Исполнитель   | Зависимости          | Критерии приемки (DoD)                                                                                                                                                                        |
|----------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------|----------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ISSUE-5.1            | <b>Интеграция ONNX-модели Style Transfer и композитинг</b><br>Даунскейл рабочей копии. Вызов инференса модели переноса стиля через ONNX Runtime (Ort::Session::Run). Апскейл результата. Наложение высокочастотных деталей оригинала на стилизованный холст для сохранения разрешения. | Разработчик 2 | ISSUE-2.3            | Применение стиля сохраняет оригинальный размер кадра. Отсутствуют сильные артефакты растягивания (pixelation). Поддерживается безопасная отмена инференса.                                    |
| ISSUE-5.2            | <b>QML-интерфейс инструмента "Стиль" (Превью-режим)</b><br>Создание горизонтального ListView с миниатюрами стилей. Реализация debounce (задержки в 300-500 мс) перед запуском тяжелого инференса, чтобы исключить запуск модели при случайных/быстрых свайпах.                         | Разработчик 4 | ISSUE-1.4, ISSUE-5.1 | Список плавно прокручивается. При выборе стиля появляется индикатор загрузки. Если пользователь быстро кликает на разные стили, в очередь становится только последний, предыдущие отменяются. |



## Слой 6: Продвинутые функции и постобработка (Дополнительные пожелания)

Этот слой включает реализацию дополнительных пожеланий заказчика, которые значительно повышают ценность продукта для пользователя, но технически возможны только после стабилизации базовой архитектуры (Слои 0–5). Реализация этих фич требует тонкой работы с QML Canvas и управления сериализацией данных.

Замена фона на пользовательское фото из галереи требует повторного вызова `Sailfish.Pickers`. Выбранное пользователем изображение может иметь любые пропорции и размер. C++ ядро должно корректно масштабировать это изображение, центрировать его и обрезать (Crop) под текущие размеры рабочего холста (Aspect Fill), после чего пропустить его через алгоритм Alpha Blending в качестве фонового слоя.

Ручная правка маски сегментации (кисть/ластик) требует внедрения интерактивного Canvas элемента поверх основного изображения в QML. Учитывая известные проблемы Canvas в различных версиях мобильных ОС (в том числе проблемы с ориентацией и цветопередачей), координаты касания (Touch) пользователя на экране должны конвертироваться в относительные координаты, а затем проецироваться на закэшированную в C++ альфа-маску полного разрешения. После отпускания пальца (`onReleased`), C++ модифицирует маску и мгновенно перерисовывает Alpha Blending.

Реализация пакетной обработки (применение одних и тех же инструментов к набору фотографий) опирается на архитектурное решение из Слойа 0 — паттерн Command. Менеджер конвейера сериализует стек выполненных команд (например: `[{"cmd": "enhance"}, {"cmd": "style", "id": "picasso"}]`) в JSON формат. При выборе нескольких фотографий создается пул фоновых задач, каждая из которых загружает снимок, десериализует JSON, последовательно вызывает нужные команды (включая последовательный инференс ONNX моделей в фоновом `BatchWorker`) без взаимодействия с UI и сохраняет результат на диск.

### Задачи Слойа 6

| Идентификатор задачи | Название и описание архитектурной задачи                                                  | Исполнитель   | Зависимости | Критерии приемки (DoD)                                                                          |
|----------------------|-------------------------------------------------------------------------------------------|---------------|-------------|-------------------------------------------------------------------------------------------------|
| ISSUE-6.1            | <b>Кастомный фото-фон из галереи</b><br>Добавление кнопки "Свое фото" в инструмент "Фон". | Разработчик 3 | ISSUE-4.3   | Изображение подложки корректно растягивается и центрируется. Нет искажений пропорций. Корректно |

| Идентификатор задачи | Название и описание архитектурной задачи                                                                                                                                                                                                                                      | Исполнитель                              | Зависимости     | Критерии приемки (DoD)                                                                                                                                                    |
|----------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------|-----------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                      | Использование <code>Sailfish.Pickers</code> для получения пути. В C++: загрузка фото, обрезка <code>AspectFill</code> под размер текущего холста, использование как подложки для Alpha Blending.                                                                              |                                          |                 | отрабатывает EXIF-ориентация подложки.                                                                                                                                    |
| ISSUE-6.2            | <b>Интерактивная правка маски (Кисть/Ластик)</b><br>Реализация <code>MouseArea</code> поверх QML Image. Сбор координат перемещения пальца. Трансляция координат в C++ методом <code>modifyMask(x, y, radius, isEraser)</code> . Модификация альфа-маски и пересчет блендинга. | Разработчик 4 (UI) + Разработчик 3 (C++) | ISSUE-4.3       | При рисовании по экрану маска в реальном времени корректируется (добавляются или удаляются области переднего плана). Точная калибровка координат при любом масштабе фото. |
| ISSUE-6.3            | <b>Смешивание стилей (Регулятор интенсивности)</b><br>Добавление ползунка "Сила стиля" (0-100%). Реализация алгоритма:<br>$Result = Original * (1.0 - Opacity) + Styled * Opacity$<br>Ползунок меняет только переменную прозрачности, не перезапуская инференс.               | Разработчик 5                            | ISSUE-5.2       | Изменение интенсивности происходит плавно (60 FPS). Кэширование стилизованного слоя работает корректно. В приложении доступно не менее 3 различных стилей.                |
| ISSUE-6.4            | <b>Сериализация проектов и пакетная обработка</b><br>Запись стека паттерна Command в локальный SQLite (или JSON файл). Создание экрана                                                                                                                                        | Разработчик 1                            | Завершен Слой 5 | Пользователь может выделить несколько фото, нажать "Применить стиль из проекта", и приложение в фоне обрабатает                                                           |

| Идентификатор задачи | Название и описание архитектурной задачи                                                                                                                     | Исполнитель | Зависимости | Критерии приемки (DoD)                               |
|----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------|-------------|------------------------------------------------------|
|                      | "История". Реализация фонового <code>BatchWorker</code> , который принимает список путей из <code>ImagePicker</code> и применяет к ним сериализованный JSON. |             |             | все фото, отображая общий <code>ProgressBar</code> . |

---

## Итог архитектурного планирования

Разделение проекта на представленные слои минимизирует технический долг и гарантирует, что базовые требования к продукту (офлайн-работа, приватность, быстродействие) будут удовлетворены с самых ранних этапов жизненного цикла разработки. Особое внимание следует уделять оптимизации работы с памятью в C++, поскольку аллокация объектов `QImage` и создание сессий/тензоров ONNX Runtime в циклах обработки приведет к неминуемым сбоям из-за нехватки памяти (OOM) на мобильных ARM-процессорах. Инкапсуляция логики ML-моделей в фоновые `QRunnable` потоки, использование возможностей конфигурации `Ort::SessionOptions` (memory arena, лимиты на потоки инференса) и грамотное использование QML-прослойки `QQuickImageProvider` обеспечит плавный и отзывчивый интерфейс, соответствующий современным стандартам операционной системы Аврора, сохраняя при этом строгую изоляцию пользовательских данных на устройстве.

---

## Источники и справочные материалы

1. **Implementation of a Neural Network Model on the Aurora Operating System Using ARM Architecture** — ResearchGate, [Ссылка](#)
2. **ONNX Runtime C++ API Reference** — Документация по интеграции среды исполнения, сессий и тензоров на C++, [Microsoft ONNX Runtime Docs](#)

3. **Qt 4.7/5/6: Using QML in C++ Applications** — Взаимодействие потоков и интеграция бизнес-логики.
4. **Пакеты Conan и CMake для разработчиков ОС Аврора** — Интеграция сторонних библиотек, [Портал разработчиков ОС Аврора](#)
5. **Файл .desktop и политики изоляции приложений** — Документация ОС Аврора по разрешениям sandbox, [Документация ОС Аврора](#)
6. **Optimizing Image Analysis for Speed on Mobile ARM CPUs** — Рекомендации по оптимизации многопоточных вычислений и работы с памятью.
7. **QQuickImageProvider Class Reference** — Реализация динамического обновления изображений в QML, [Qt Documentation](#)
8. **Canvas.requestPaint() и Canvas.loadImage в мобильных окружениях Qt** — Известные проблемы с цветовыми каналами и метаданными EXIF, обсуждения на форумах Sailfish OS.